Day 15

Advanced SQL Practice

Aggregates · GROUP BY & HAVING · Joins · Subqueries · Relationships · Indexes · Normalization

#	Section	What you practice
1	Aggregate Functions	COUNT, SUM, AVG, MIN, MAX
2	GROUP BY and HAVING	Group records and filter grouped results
3	Joins	Student and course data across related tables
4	Subqueries	IN, EXISTS, correlated, derived tables
5	Relationships	Primary keys, foreign keys, many-to-many
6	Index Basics	Speeding up lookups and joins
7	Normalization	Spotting and removing duplicate data
8	Challenges + Answer Key	Practice on your own, then check

How to use this PDF

- Read the key ideas box at the start of each section.
- Cover the result table, read the query, and predict the output before checking.
- Copy the setup script into SQLite, MySQL or PostgreSQL (see the .sql file) and run each query yourself.
- Change a condition or a column in each query and see how the result changes.
- Finish with the challenge questions before opening the Answer Key.

Tables and relationships

```
One-to-many:   instructors -> courses   (one instructor teaches many courses)
Many-to-many:  students <-> courses     (resolved by the enrollments table)
```

```sql
CREATE TABLE students (
    student_id  INTEGER PRIMARY KEY,
    name        VARCHAR(50)  NOT NULL,
    email       VARCHAR(100) NOT NULL UNIQUE,
    city        VARCHAR(50),
    age         INTEGER
);

CREATE TABLE instructors (
    instructor_id INTEGER PRIMARY KEY,
    name          VARCHAR(50) NOT NULL
);

CREATE TABLE courses (
    course_id     INTEGER PRIMARY KEY,
    title         VARCHAR(100) NOT NULL,
    fee           INTEGER NOT NULL,
    instructor_id INTEGER,
    FOREIGN KEY (instructor_id) REFERENCES instructors(instructor_id)
);
```

```sql
CREATE TABLE enrollments (
    enrollment_id INTEGER PRIMARY KEY,
    student_id    INTEGER NOT NULL,
    course_id     INTEGER NOT NULL,
    marks         INTEGER,              -- NULL = not graded yet
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)
);
```

Sample data

students

student_id	name	email	city	age
1	Aarav	aarav@mail.com	Pune	21
2	Diya	diya@mail.com	Mumbai	22
3	Rohan	rohan@mail.com	Aurangabad	20
4	Sneha	sneha@mail.com	Pune	23
5	Karan	karan@mail.com	Nagpur	21
6	Meera	meera@mail.com	Mumbai	24

instructors

instructor_id	name
1	Dr. Rao
2	Ms. Iyer
3	Mr. Khan

courses

course_id	title	fee	instructor_id
101	SQL Basics	4000	1
102	Python	6000	2
103	Web Development	8000	2
104	Data Science	12000	1
105	Cloud Computing	9000	NULL

enrollments

enrollment_id	student_id	course_id	marks
1	1	101	85
2	1	102	78
3	2	101	92
4	2	103	88
5	2	104	95
6	3	102	60
7	3	101	NULL
8	4	104	72
9	4	102	81
10	5	103	55

1. Aggregate Functions

KEY IDEAS

- Aggregates collapse many rows into one value.
- **COUNT(*)** counts rows; **COUNT(col)** skips NULLs. SUM, AVG, MIN and MAX also ignore NULLs.
- Use **COUNT(DISTINCT col)** to count unique values.

1.1 Total students

```
SELECT COUNT(*) AS total_students FROM students;
```

Result (1 row)

total_students
6

1.2 COUNT(*) vs COUNT(column): COUNT(column) ignores NULLs

```
SELECT COUNT(*)     AS total_enrollments,
       COUNT(marks) AS graded_enrollments
FROM enrollments;
```

Result (1 row)

total_enrollments	graded_enrollments
10	9

1.3 Revenue potential: SUM of all course fees

```
SELECT SUM(fee) AS total_fees FROM courses;
```

Result (1 row)

total_fees
39000

1.4 Average, highest and lowest marks (NULLs are skipped)

```
SELECT AVG(marks) AS avg_marks,
       MAX(marks) AS highest,
       MIN(marks) AS lowest
FROM enrollments;
```

Result (1 row)

avg_marks	highest	lowest
78.44	95	55

1.5 Rounded average

```
SELECT ROUND(AVG(marks), 2) AS avg_marks FROM enrollments;
```

Result (1 row)

avg_marks
78.44

1.6 Cheapest and most expensive course

```
SELECT MIN(fee) AS cheapest, MAX(fee) AS costliest FROM courses;
```

Result (1 row)

cheapest	costliest
4000	12000

1.7 Count distinct cities

```sql
SELECT COUNT(DISTINCT city) AS unique_cities FROM students;
```

Result (1 row)

unique_cities
4

2. GROUP BY and HAVING

KEY IDEAS

- **GROUP BY** returns one row per group; every non-aggregated SELECT column must be in GROUP BY.
- **WHERE** filters rows before grouping; **HAVING** filters groups after aggregation.
- Logical order: FROM > WHERE > GROUP BY > HAVING > SELECT > ORDER BY.

```
WHERE  filters ROWS   before grouping.
HAVING filters GROUPS after grouping (can use aggregates).
```

2.1 Students per city

```sql
SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city;
```

Result (4 rows)

city	student_count
Aurangabad	1
Mumbai	2
Nagpur	1
Pune	2

2.2 Cities with more than one student

```sql
SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city
HAVING COUNT(*) > 1;
```

Result (2 rows)

city	student_count
Mumbai	2
Pune	2

2.3 Number of enrollments per course

```sql
SELECT course_id, COUNT(*) AS enrolled
FROM enrollments
GROUP BY course_id
ORDER BY enrolled DESC;
```

Result (4 rows)

course_id	enrolled
102	3
101	3
104	2
103	2

2.4 Average marks per course, only courses averaging above 75

```sql
SELECT course_id, ROUND(AVG(marks), 1) AS avg_marks
FROM enrollments
GROUP BY course_id
HAVING AVG(marks) > 75;
```

Result (2 rows)

course_id	avg_marks
101	88.5
104	83.5

2.5 WHERE + GROUP BY + HAVING together:

among graded rows only, courses with 2+ graded students

```sql
SELECT course_id, COUNT(*) AS graded_students, AVG(marks) AS avg_marks
FROM enrollments
WHERE marks IS NOT NULL
GROUP BY course_id
HAVING COUNT(*) >= 2
ORDER BY avg_marks DESC;
```

Result (4 rows)

course_id	graded_students	avg_marks
101	2	88.5
104	2	83.5
102	3	73.0
103	2	71.5

2.6 Total marks per student, only students above 150 total

```sql
SELECT student_id, SUM(marks) AS total_marks
FROM enrollments
GROUP BY student_id
HAVING SUM(marks) > 150;
```

Result (3 rows)

student_id	total_marks
1	163
2	275
4	153

3. Joins

KEY IDEAS

- **INNER JOIN** keeps only matching rows; **LEFT JOIN** keeps every left row (NULL where no match).
- LEFT JOIN plus **WHERE right_key IS NULL** finds rows with no match (students with no courses).
- Join on keys (PK = FK) and use table aliases to avoid ambiguous column names.

3.1 INNER JOIN: student name + course title + marks

(only rows that match in all three tables)

```sql
SELECT s.name AS student, c.title AS course, e.marks
FROM enrollments e
INNER JOIN students s ON e.student_id = s.student_id
INNER JOIN courses  c ON e.course_id  = c.course_id
ORDER BY s.name, c.title;
```

Result (10 rows)

student	course	marks
Aarav	Python	78
Aarav	SQL Basics	85
Diya	Data Science	95
Diya	SQL Basics	92
Diya	Web Development	88
Karan	Web Development	55
Rohan	Python	60
Rohan	SQL Basics	NULL
Sneha	Data Science	72
Sneha	Python	81

3.2 LEFT JOIN: ALL students, even those with no enrollments

(Meera appears with NULL course)

```
SELECT s.name AS student, c.title AS course
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
LEFT JOIN courses     c ON e.course_id  = c.course_id
ORDER BY s.name;
```

Result (11 rows)

student	course
Aarav	SQL Basics
Aarav	Python
Diya	SQL Basics
Diya	Web Development
Diya	Data Science
Karan	Web Development
Meera	NULL
Rohan	SQL Basics
Rohan	Python
Sneha	Python
Sneha	Data Science

3.3 Find students with NO enrollments (LEFT JOIN + IS NULL)

```
SELECT s.name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
WHERE e.enrollment_id IS NULL;
```

Result (1 row)

name
Meera

3.4 Courses nobody has enrolled in

```
SELECT c.title
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
WHERE e.enrollment_id IS NULL;
```

Result (1 row)

title
Cloud Computing

3.5 Courses with their instructor (LEFT JOIN keeps "Cloud Computing")

```
SELECT c.title, COALESCE(i.name, 'Not assigned') AS instructor
FROM courses c
LEFT JOIN instructors i ON c.instructor_id = i.instructor_id;
```

Result (5 rows)

title	instructor
SQL Basics	Dr. Rao
Python	Ms. Iyer
Web Development	Ms. Iyer
Data Science	Dr. Rao
Cloud Computing	Not assigned

3.6 JOIN + GROUP BY: students per course (with title)

```
SELECT c.title, COUNT(e.student_id) AS students_enrolled
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
ORDER BY students_enrolled DESC;
```

Result (5 rows)

title	students_enrolled
SQL Basics	3
Python	3
Web Development	2
Data Science	2
Cloud Computing	0

3.7 JOIN + GROUP BY + HAVING: students taking 2+ courses

```
SELECT s.name, COUNT(*) AS courses_taken
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
HAVING COUNT(*) >= 2;
```

Result (4 rows)

name	courses_taken
Aarav	2
Diya	3
Rohan	2
Sneha	2

3.8 Total fees each student has to pay

```sql
SELECT s.name, SUM(c.fee) AS total_fee
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c     ON e.course_id  = c.course_id
GROUP BY s.student_id, s.name
ORDER BY total_fee DESC;
```

Result (5 rows)

name	total_fee
Diya	24000
Sneha	18000
Aarav	10000
Rohan	10000
Karan	8000

3.9 Average marks per instructor

```sql
SELECT i.name AS instructor, ROUND(AVG(e.marks), 1) AS avg_marks
FROM instructors i
JOIN courses c     ON c.instructor_id = i.instructor_id
JOIN enrollments e ON e.course_id     = c.course_id
GROUP BY i.instructor_id, i.name;
```

Result (2 rows)

instructor	avg_marks
Dr. Rao	86.0
Ms. Iyer	72.4

3.10 SELF JOIN: pairs of students living in the same city

```sql
SELECT a.name AS student_1, b.name AS student_2, a.city
FROM students a
JOIN students b
  ON a.city = b.city AND a.student_id < b.student_id;
```

Result (2 rows)

student_1	student_2	city
Aarav	Sneha	Pune
Diya	Meera	Mumbai

4. Subqueries

KEY IDEAS

- A subquery is a query inside another query: in WHERE, FROM or SELECT.
- A **correlated** subquery refers to the outer row and is evaluated once per outer row.
- Prefer **NOT EXISTS** over NOT IN when the subquery can return NULL.

4.1 Scalar subquery: students older than the average age

```
SELECT name, age
FROM students
WHERE age > (SELECT AVG(age) FROM students);
```

Result (3 rows)

name	age
Diya	22
Sneha	23
Meera	24

4.2 Course with the highest fee

```
SELECT title, fee
FROM courses
WHERE fee = (SELECT MAX(fee) FROM courses);
```

Result (1 row)

title	fee
Data Science	12000

4.3 IN: students enrolled in "SQL Basics"

```
SELECT name
FROM students
WHERE student_id IN (
    SELECT e.student_id
    FROM enrollments e
    JOIN courses c ON e.course_id = c.course_id
    WHERE c.title = 'SQL Basics'
);
```

Result (3 rows)

name
Aarav
Diya
Rohan

4.4 NOT IN: students who never enrolled

```
SELECT name
FROM students
WHERE student_id NOT IN (SELECT student_id FROM enrollments);
```

Result (1 row)

name
Meera

4.5 EXISTS: courses that have at least one enrollment

```
SELECT c.title
FROM courses c
WHERE EXISTS (SELECT 1 FROM enrollments e WHERE e.course_id = c.course_id);
```

Result (4 rows)

title
SQL Basics
Python
Web Development
Data Science

4.6 Correlated subquery: each student's marks above THEIR OWN average

```
SELECT e.student_id, e.course_id, e.marks
FROM enrollments e
WHERE e.marks > (
    SELECT AVG(e2.marks)
    FROM enrollments e2
    WHERE e2.student_id = e.student_id
);
```

Result (4 rows)

student_id	course_id	marks
1	101	85
2	101	92
2	104	95
4	102	81

4.7 Subquery in FROM (derived table): average of per-student totals

```
SELECT AVG(total) AS avg_of_student_totals
FROM (
    SELECT student_id, SUM(marks) AS total
    FROM enrollments
    GROUP BY student_id
) AS t;
```

Result (1 row)

avg_of_student_totals
141.2

4.8 Subquery in SELECT: each course with its enrollment count

```
SELECT c.title,
       (SELECT COUNT(*) FROM enrollments e WHERE e.course_id = c.course_id) AS enrolled
FROM courses c;
```

Result (5 rows)

title	enrolled
SQL Basics	3
Python	3
Web Development	2
Data Science	2
Cloud Computing	0

4.9 Second-highest fee

```sql
SELECT MAX(fee) AS second_highest_fee
FROM courses
WHERE fee < (SELECT MAX(fee) FROM courses);
```

Result (1 row)

second_highest_fee
9000

4.10 Top scorer: student with the highest single mark

```sql
SELECT s.name, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
WHERE e.marks = (SELECT MAX(marks) FROM enrollments);
```

Result (1 row)

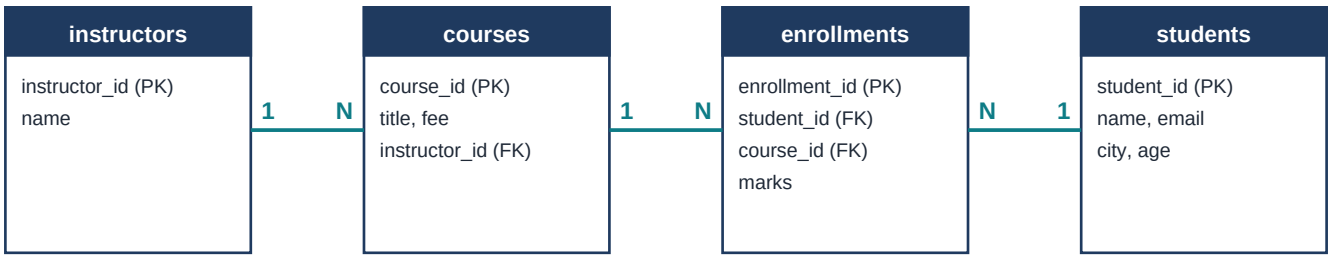
name	marks
Diya	95

```
NOTE: NOT IN returns nothing if the subquery contains a NULL.
      Prefer NOT EXISTS or LEFT JOIN ... IS NULL when NULLs are possible.
```

5. Relationships Between Tables

KEY IDEAS

- A **primary key** uniquely identifies a row; a **foreign key** points to a primary key elsewhere and keeps data valid.
- Many-to-many relationships need a junction table (here: enrollments).



PK = primary key FK = foreign key students <-> courses is many-to-many via enrollments

```
instructors (1) ----< (many) courses          one-to-many
students    (1) ----< (many) enrollments      one-to-many
courses     (1) ----< (many) enrollments      one-to-many
students (many) >----< (many) courses         many-to-many via enrollments

PRIMARY KEY  = uniquely identifies a row
FOREIGN KEY  = points to a primary key in another table (keeps data valid)
```

Check referential integrity: enrollments pointing to missing students (should return 0 rows)

```sql
SELECT e.*
FROM enrollments e
LEFT JOIN students s ON e.student_id = s.student_id
WHERE s.student_id IS NULL;
```

Result (0 rows)

(no rows returned)

```
View table structure
  SQLite:     PRAGMA table_info(students);
  MySQL:      DESCRIBE students;
  PostgreSQL: \d students
```

6. Index Basics

KEY IDEAS

- Indexes speed up lookups, joins and sorting on the indexed columns.
- Cost: extra storage and slower INSERT/UPDATE/DELETE. Primary key and UNIQUE columns are indexed automatically.
- Index columns used in WHERE, JOIN and ORDER BY, not every column.

```
An index is like a book's index: it lets the database find rows without
scanning the whole table. Primary keys and UNIQUE columns are indexed
automatically.

Good candidates: columns used in WHERE, JOIN ... ON, ORDER BY.
Trade-off: faster reads, slightly slower INSERT/UPDATE/DELETE + extra space.
```

```
CREATE INDEX idx_enrollments_student ON enrollments(student_id);
CREATE INDEX idx_enrollments_course  ON enrollments(course_id);
CREATE INDEX idx_students_city       ON students(city);
```

Composite index (order matters: leftmost column first)

```
CREATE INDEX idx_enroll_student_course ON enrollments(student_id, course_id);
```

Prevent a student enrolling twice in the same course

```
CREATE UNIQUE INDEX uq_student_course ON enrollments(student_id, course_id);
```

```
See whether the index is used
  SQLite:        EXPLAIN QUERY PLAN SELECT * FROM students WHERE city = 'Pune';
  MySQL/Postgres: EXPLAIN SELECT * FROM students WHERE city = 'Pune';
```

```
Remove an index
DROP INDEX idx_students_city;          -- SQLite / PostgreSQL
DROP INDEX idx_students_city ON students;  -- MySQL
```

7. Normalization Fundamentals

KEY IDEAS

- Normalization = store each fact once, in one place.
- Duplicated data causes update, insert and delete anomalies.
- Find duplicates with **GROUP BY ... HAVING COUNT(*) > 1**.

```
A BAD (denormalized) table: everything crammed together, with repeats.
```

```sql
CREATE TABLE student_course_raw (
    id            INTEGER PRIMARY KEY,
    student_name  VARCHAR(50),
    student_email VARCHAR(100),
    student_city  VARCHAR(50),
    course_title  VARCHAR(100),
    course_fee    INTEGER,
    instructor    VARCHAR(50)
);

INSERT INTO student_course_raw VALUES
(1, 'Aarav', 'aarav@mail.com', 'Pune',   'SQL Basics', 4000, 'Dr. Rao'),
(2, 'Aarav', 'aarav@mail.com', 'Pune',   'Python',     6000, 'Ms. Iyer'),
(3, 'Diya',  'diya@mail.com',  'Mumbai', 'SQL Basics', 4000, 'Dr. Rao'),
(4, 'Diya',  'diya@mail.com',  'Mumbai', 'SQL Basics', 4000, 'Dr. Rao'),  -- exact duplicate
(5, 'Rohan', 'rohan@mail.com', 'Pune',   'Python',     6000, 'Ms. Iyer');
```

Contents of student_course_raw (note the repeats)

id	student_name	student_email	student_city	course_title	course_fee	instructor
1	Aarav	aarav@mail.com	Pune	SQL Basics	4000	Dr. Rao
2	Aarav	aarav@mail.com	Pune	Python	6000	Ms. Iyer
3	Diya	diya@mail.com	Mumbai	SQL Basics	4000	Dr. Rao
4	Diya	diya@mail.com	Mumbai	SQL Basics	4000	Dr. Rao
5	Rohan	rohan@mail.com	Pune	Python	6000	Ms. Iyer

7.1 Find exact duplicate rows (same student + same course)

```sql
SELECT student_email, course_title, COUNT(*) AS times_repeated
FROM student_course_raw
GROUP BY student_email, course_title
HAVING COUNT(*) > 1;
```

Result (1 row)

student_email	course_title	times_repeated
diya@mail.com	SQL Basics	2

7.2 Find repeated values that signal redundancy

(student details stored once per course; course details once per student)

```sql
SELECT student_email, COUNT(*) AS rows_for_student
FROM student_course_raw
GROUP BY student_email
HAVING COUNT(*) > 1;
```

Result (2 rows)

student_email	rows_for_student
aarav@mail.com	2
diya@mail.com	2

```sql
SELECT course_title, course_fee, instructor, COUNT(*) AS repeats
FROM student_course_raw
GROUP BY course_title, course_fee, instructor
HAVING COUNT(*) > 1;
```

Result (2 rows)

course_title	course_fee	instructor	repeats
Python	6000	Ms. Iyer	2
SQL Basics	4000	Dr. Rao	3

7.3 Delete duplicates, keeping the lowest id of each group

```sql
DELETE FROM student_course_raw
WHERE id NOT IN (
    SELECT keep_id FROM (
        SELECT MIN(id) AS keep_id
        FROM student_course_raw
        GROUP BY student_email, course_title
    ) AS k
);
```

student_course_raw after the delete (id 4 removed)

id	student_name	student_email	student_city	course_title	course_fee	instructor
1	Aarav	aarav@mail.com	Pune	SQL Basics	4000	Dr. Rao
2	Aarav	aarav@mail.com	Pune	Python	6000	Ms. Iyer
3	Diya	diya@mail.com	Mumbai	SQL Basics	4000	Dr. Rao
5	Rohan	rohan@mail.com	Pune	Python	6000	Ms. Iyer

7.4 Check for duplicates in the main tables

```sql
SELECT email, COUNT(*) FROM students GROUP BY email HAVING COUNT(*) > 1;
```

Result (0 rows)

(no rows returned)

Normal forms in short

```
1NF: every cell holds ONE value (no "Python, SQL" lists), each row unique.
2NF: 1NF + no column depends on only PART of a composite key.
     (course_fee depends on course, not on student+course -> split it)
3NF: 2NF + non-key columns depend ONLY on the key, not on other
     non-key columns (instructor depends on course, not on the row id).

The raw table above breaks 2NF/3NF:
  * student_city repeats for every course the student takes
  * course_fee and instructor repeat for every student in the course
  * Update anomaly: change Aarav's city in one row and forget another
  * Delete anomaly: removing the last student of a course loses the course

FIX: split into students, instructors, courses, enrollments
     -> exactly the structure created in section 0.
```

7.5 Migrate the cleaned raw data into the normalized tables (example)

```sql
-- INSERT INTO students (name, email, city)
--     SELECT DISTINCT student_name, student_email, student_city FROM student_course_raw;
```

8. Challenge Questions

KEY IDEAS

- Try each question on your own first. Solutions are in the Answer Key at the end of this PDF.

```
Q1. List each student with their average marks, highest first.
Q2. Which course earns the most revenue (fee x number of enrollments)?
Q3. Names of students whose total marks are above the overall average
    of student totals.
Q4. Instructors who teach no course (hint: LEFT JOIN ... IS NULL).
Q5. Cities where the average student age is above 21.
Q6. Students who scored the maximum mark in at least one course
    (hint: correlated subquery on course_id).
Q7. Add an index to speed up Q1 and explain why it helps.
```

Answer Key: Challenge Questions

Q1

```sql
SELECT s.name, ROUND(AVG(e.marks), 1) AS avg_marks
FROM students s JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
ORDER BY avg_marks DESC;
```

Result (5 rows)

name	avg_marks
Diya	91.7
Aarav	81.5
Sneha	76.5
Rohan	60.0
Karan	55.0

Q2

```sql
SELECT c.title, c.fee * COUNT(e.enrollment_id) AS revenue
FROM courses c JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title, c.fee
ORDER BY revenue DESC;
```

Result (4 rows)

title	revenue
Data Science	24000
Python	18000
Web Development	16000
SQL Basics	12000

Q3

```sql
SELECT s.name, SUM(e.marks) AS total
FROM students s JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name
HAVING SUM(e.marks) > (
    SELECT AVG(total) FROM (
        SELECT SUM(marks) AS total FROM enrollments GROUP BY student_id
    ) AS t
);
```

Result (3 rows)

name	total
Aarav	163
Diya	275
Sneha	153

Q4

```sql
SELECT i.name
FROM instructors i LEFT JOIN courses c ON c.instructor_id = i.instructor_id
WHERE c.course_id IS NULL;
```

Result (1 row)

name
Mr. Khan

Q5

```sql
SELECT city, AVG(age) AS avg_age FROM students GROUP BY city HAVING AVG(age) > 21;
```

Result (2 rows)

city	avg_age
Mumbai	23.0
Pune	22.0

Q6

```sql
SELECT DISTINCT s.name
FROM students s JOIN enrollments e ON s.student_id = e.student_id
WHERE e.marks = (SELECT MAX(e2.marks) FROM enrollments e2 WHERE e2.course_id = e.course_id);
```

Result (2 rows)

name
Diya
Sneha

Q7

Q1 groups and joins on student_id, so index that column. The database can then look up each student's enrollments directly instead of scanning the whole table for every student.

```sql
CREATE INDEX idx_enrollments_student ON enrollments(student_id);
```

Quick Reference Cheat Sheet

Join types

Join	Returns
INNER JOIN	Only rows with a match in both tables
LEFT JOIN	All rows from the left table, plus matches from the right (NULL if none)
RIGHT JOIN	All rows from the right table, plus matches from the left
FULL JOIN	All rows from both tables (not supported in MySQL or older SQLite)
SELF JOIN	A table joined to itself using two aliases (e.g. students in the same city)

Aggregate functions

Function	Meaning	NULL handling
COUNT(*)	Number of rows	Counts every row
COUNT(col)	Number of non-NULL values	Skips NULLs
SUM(col) / AVG(col)	Total / average	Skips NULLs
MIN(col) / MAX(col)	Smallest / largest value	Skips NULLs

Order in which SQL evaluates a query

Step	Clause	Purpose
1	FROM / JOIN	Pick and combine tables
2	WHERE	Filter rows
3	GROUP BY	Form groups
4	HAVING	Filter groups
5	SELECT	Compute output columns
6	ORDER BY / LIMIT	Sort and trim

Normal forms

Form	Rule
1NF	One value per cell, no repeating groups, every row unique
2NF	1NF + every non-key column depends on the whole key
3NF	2NF + non-key columns depend only on the key, not on each other

Common mistakes

- Using an aggregate in WHERE (use HAVING).
- Selecting a column that is not in GROUP BY and not aggregated.
- Forgetting the ON condition in a join, which produces a cartesian product.
- Using NOT IN with a subquery that can return NULL (the result is empty). Use NOT EXISTS instead.
- Confusing COUNT(*) with COUNT(column) when NULLs are present.
- Filtering the right table in WHERE after a LEFT JOIN, which turns it into an inner join. Put that condition in ON.